מבוא לאבטחת סייבר · 689156 · המכללה למינהל

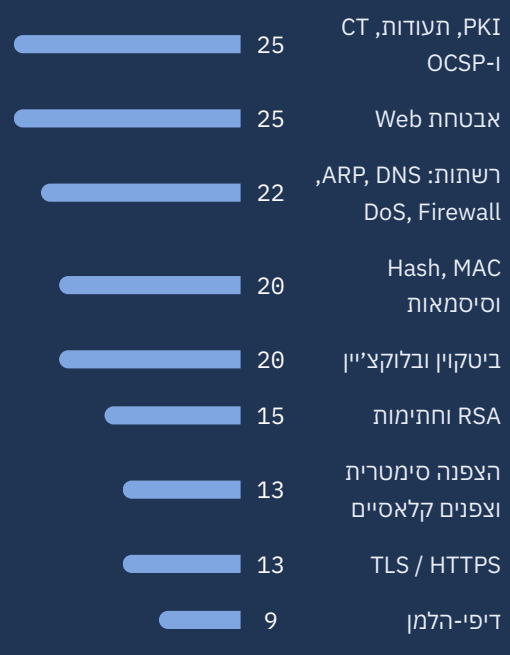
חוברת סיכום לקראת המבחן

כל חומר הקורס בתמצית, מוצלב עם 162 השאלות שהופיעו במבחנים בין 2022 ל-2026. הדגש על תיאוריה וחישובים. בסוף: מלכודות שחוזרות, נוסחאות, 17 דוגמאות חישוב פתורות ומילון מושגים.

איך לעבוד עם החוברת

1. קוראים פרק, ואז פותרים את שאלות הנושא שלו באפליקציית התרגול. מתקשים בנושא? יש סרטון הסבר בספריית הסרטונים.
2. כל טעות: חוזרים לתיבות **במבחן** ו-**מלכודת** של הנושא.
3. יום לפני המבחן: פרק "טענות שחוזרות", דף הנוסחאות, הדוגמאות הפתורות והמילון.

משקל הנושאים במאגר השאלות



מבנה המבחן: 10 שאלות אמריקאיות, שעה ועשרים דקות (8 דקות לשאלה), בלי חומר עזר. אין קוד: השאלות הן תיאוריה וחישובים מתמטיים. שאלות קוד ממבחני עבר מופיעות כאן כשאלות תיאוריה.

תוכן העניינים

1 מבוא, מודלי תוקף ותקשורת ברשת

2 הצפנה סימטרית וצפנים קלאסיים

3 פונקציות Hash, סיסמאות ו-MAC

4 קריפטוגרפיה אסימטרית: דיפי-הלמן, RSA וחתימות

5 PKI ותעודות דיגיטליות

6 תקשורת מאובטחת: TLS ו-HTTPS

7 ביטקוין ובלוקצ'יין

8 אבטחת Web

9 אבטחת רשת: ARP, DNS, DoS וחומות אש

10 טענות שחוזרות במבחנים

11 דף נוסחאות ודוגמאות פתורות

12 מילון מושגים

הסילבוס מול החוברת

נושא בסילבוס	מה כולל	פרק
מבוא	עקרונות, דוגמאות מהתקפות עבר, מודלי תוקף (off-path, MitM)	1
קריפטוגרפיה	צפנים היסטוריים, הצפנה סימטרית וא-סימטרית, אימות וחתימות, hash	2, 3, 4
Cryptocurrency and blockchains	ביטקוין: יסודות, טכנולוגיה, קריפטוגרפיה; בלוקצ'יין	7
תקשורת מאובטחת	PKI, מנגנוני החלפת מפתחות, TLS	4, 5, 6
הגנה ב-web	עוגיות, SOP, SQL injection, XSS, XSRF, clickjacking	8
הגנה בשכבות אחרות	סריקות, sniffing, spoofing, הרעלת ARP ו-Kali, DNS, TCP, DoS/DDoS, Botnets	1, 9

1 מבוא, מודלי תוקף ותקשורת ברשת

למה אבטחת סייבר קשה

תוקפים מונעים בעיקר מכסף (כופרה כמו WannaCry ו-CryptoLocker, גניבת מידע, סחיטה), ממניעים מדינתיים (תשתיות קריטיות) או מרצון להביך. בצד המגן אין תמריץ כלכלי חזק לכתוב קוד מאובטח, אבטחה יקרה ואיטית, ויש גורם אנושי וספריות חיצוניות עם חולשות.

כתובות ורכיבי רשת

- **כתובת MAC**: כתובת פיזית בת 48 ביט (12 ספרות הקסדצימליות), צרובה בכרטיס הרשת. לא תלויה במיקום (אפשר לזייף אותה).
- **כתובת IP**: כתובת לוגית שתלויה ברשת שאליה מחוברים. למחשבים באותה רשת יש תחילית משותפת (למשל 192.168.0.x).
- **Hub**: משכפל כל חבילה לכל הפורטים (broadcast). כל מי שמחובר אליו יכול להאזין.
- **Switch**: מחזיק טבלת CAM שממפה MAC לפורט פיזי, ושולח כל חבילה רק ליעד. אם היעד לא בטבלה, הוא מתנהג כמו hub.
- **Gateway**: הנתב שמעביר תעבורה מהרשת המקומית החוצה לאינטרנט.

שלושת מודלי התוקף

מודל	מה יכול	מה לא יכול
Off-path	לשלוח הודעות משלו, לזייף כתובות מקור (IP/MAC spoofing)	לראות את התעבורה בין הצדדים, לשנות או לחסום אותה
Eavesdropper (מאזין)	כל מה ש-off-path יכול, ובנוסף לקבל עותק של כל הודעה	לשנות, לחסום או לעכב הודעות
MitM	הכל: לקרוא, לשנות, לחסום, לעכב ולהזריק. התעבורה עוברת דרכו	לשבור קריפטוגרפיה טובה

במבחן
כמעט כל שאלה על התקפה שואלת "איזה תוקף נדרש". כלל אצבע: ככל שהתוקף צריך לראות את התשובה האמיתית או לחסום אותה, הוא צריך יכולת גבוהה יותר. הרעלת ARP אפשרית כבר ל-off-path; הרעלת DNS לא מחייבת MitM.

התקפות על ה-Switch

MAC flooding: התוקף מציף את המתג בחבילות עם המון כתובות MAC מזויפות. טבלת ה-CAM מתמלאת, והמתג "משונמך" (downgrade) להתנהגות של hub ששולח לכולם. כך התוקף מקבל יכולת האזנה. הגנה: **port security** (הגבלת כמות ה-MAC לפורט). לכן הטענה "אי אפשר למנוע התקפת שנמוך נגד switch" שגויה.

עקרון קרקהוף (Kerckhoffs)

מערכת קריפטוגרפית צריכה להיות בטוחה גם כשכל פרטיה ידועים, חוץ מהמפתח. לכן משתמשים באלגוריתמים ידועים ונבדקים ולא ממציאים צופן "סודי" (security by obscurity).

2 הצפנה סימטרית וצפנים קלאסיים

בהצפנה סימטרית אותו מפתח k משמש להצפנה ולפענוח: $c = E_k(m)$, $m = D_k(c)$. דרישת נכונות: $D_k(E_k(m)) = m$. הבעיה הגדולה: איך שני הצדדים מסכימים על המפתח (פרק 4).

צפנים היסטוריים

צופן	איך עובד	מרחב מפתחות	איך שוברים
קיסר	הזזה קבועה ב-3: $c[i] = (m[i] + 3) \bmod 26$	מפתח אחד (קבוע)	יודעים את השיטה, אז שבור מיד
הזזה (shift)	$(m[i]+k) \bmod 26$. קיסר הוא מקרה פרטי עם $k = 3$	25 (או 26 כולל 0)	Brute force (חיפוש ממצה)
החלפה מונואלפביתית	כל אות ממופה לאות אחרת לפי תמורה	$26! \approx 4 \cdot 10^{26}$	ניתוח שכיחויות (E הכי שכיחה באנגלית)
Vigenère	מפתח קצר משורשר לאורך הטקסט; כל תו מוזז לפי אות המפתח שמעליו	$26^{\lvert k \rvert}$	1) מוצאים את אורך המפתח לפי מרחקים בין רצפים שחוזרים (כפולות של האורך); 2) ניתוח שכיחויות לכל עמודה בנפרד

במבחן

- קיסר על QUIZ: כל אות +3, כלומר Q→T, U→X, I→L, Z→C, ולכן TXLC. לא לשכוח את הגלישה Z→C.
- צופן החלפה על א"ב של 28 תווים, ו-ciphertext של 7 תווים זהים: כל ה-plaintext חייב להיות אותה אות 7 פעמים, ולכן יש **28** אפשרויות.
- בצופן החלפה התוקף יודע ש-X מחליף את Y: נשארו 25 אותיות לסדר, כלומר $25!$ מפתחות.
- Vigenère עם מפתח CODE (אורך 4) ו-plaintext באורך 5: תווים 1 ו-5 מוצפנים עם אותה אות מפתח, לכן הם זהים ב-ciphertext רק אם הם זהים ב-plaintext. ההסתברות: **1/26**.

(OTP) One-Time Pad

$E_k(m) = m \oplus k$, $D_k(c) = c \oplus k$. הנכונות נובעת מ-$k \oplus k = 0$. שני תנאים: $|k| \geq |m|$, והמפתח אקראי ומשמש **פעם אחת בלבד**. בתנאים האלה ה-OTP בטוח לחלוטין (perfect secrecy): ה-ciphertext לא מגלה כלום על ה-plaintext, ולכן הבטיחות **לא** תלויה בתוכן הטקסט.

החיסרון: העברת המפתח. מפתח באורך ההודעה צריך להגיע בדרך מאובטחת, ואם יש דרך כזו אפשר לשלוח בה את ההודעה עצמה. לכן "בעיה גדולה ב-OTP היא העברת המפתחות" היא טענה **נכונה**.

Two-Time Pad: שימוש חוזר במפתח נותן $c_1 \oplus c_2 = m_1 \oplus m_2$. המפתח נעלם, ותוקף מנחש מילים שכיחות (the, and) ומשחזר חלקים מהטקסטים.

מלכודת

פרוטוקול "שלוש העברות עם XOR": אליס שולחת $K_1 = K \oplus K_A$, בוב מחזיר $K_2 = K_1 \oplus K_B$, אליס שולחת $K_3 = K_2 \oplus K_A$. נראה בטוח, אבל מאזין שרואה את שלושתם מחשב $K_1 \oplus K_2 \oplus K_3 = K$. הפרוטוקול **לא מוגן אפילו מפני מאזין**.

צפני זרם וצפני בלוקים

- **PRG** (מחולל פסאודו-אקראי): לוקח seed קצר ומפיק רצף ארוך שנראה אקראי.
- **Stream cipher**: $c = m \oplus PRG(k)$. מהיר מאוד (דוגמה: RC4).
- **Block cipher**: מצפין בלוקים בגודל קבוע (DES: 64 ביט = 8 בתים; AES: 128 ביט). לטקסט ארוך צריך **mode of operation**, ולכן גם ב-AES צריך להגדיר mode.

Mode	נוסחה	תכונה
ECB	$C_i = E_k(P_i)$	בלוקים זהים מוצפנים לבלוקים זהים, ולכן דפוסים דולפים. ריפוד (padding) לבלוק האחרון
CBC	$C_i = E_k(C_{i-1} \oplus P_i),\ C_0 = IV$	בלוקים זהים נותנים פלט שונה. ה-IV אקראי ולא סודי. נותן סודיות, לא אמינות

במבחן
הצפנת ABCDEFGHABCDEFGH ב-DES-ECB: שני בלוקים זהים של 8 בתים, ולכן ה-ciphertext הוא אותו חצי פעמיים: `437d7e60707604f3 437d7e60707604f3`. מחפשים בתשובות את המחרוזת שבה החצי הראשון שווה לשני.

DES, 3DES ו-AES

- **DES**: צופן בלוקים סימטרי, בלוק 64 ביט, מפתח 56 ביט בלבד (חלש היום ל-brute force).
- **3DES** עם שלושה מפתחות במבנה **EDE**: $C = E_{K3}(D_{K2}(E_{K1}(m)))$. אם `K1 = K2 = K3`, הפענוח האמצעי מבטל את ההצפנה הראשונה, והתוצאה היא DES רגיל. זו **התמיכה לאחור**.
- עם מפתחות שונים, EEE ו-EDE נותנים פלט **שונה**. הטענה "אין הבדל" שגויה.
- **AES**: הסטנדרט המודרני, צופן בלוקים סימטרי. אלגוריתמים סימטריים במבחן: DES, 3DES, AES, RC4. RSA ו-Diffie-Hellman אסימטריים, ו-SHA256 הוא hash (לא הצפנה).

3 פונקציות Hash, סיסמאות ו-MAC

פונקציית Hash

פונקציה חד-כיוונית שמקבלת קלט בכל אורך ומחזירה פלט (digest) באורך קבוע. SHA-256 מחזירה 256 ביט. שינוי של ביט אחד בקלט משנה את הפלט לחלוטין. שימוש קלאסי: בדיקה שקובץ שהורד (למשל ISO של Ubuntu) לא שונה.

תכונה	הגדרה (כך היא מופיעה במבחן)
Pre-image resistance (חד-כיווניות)	בהינתן digest d של הודעה אקראית, קשה למצוא m כך ש-$h(m) = d$
Second pre-image resistance	בהינתן הודעה אקראית m_0, קשה למצוא $m_1 \neq m_0$ כך ש-$h(m_0) = h(m_1)$
Collision resistance	קשה למצוא **שתי** הודעות כלשהן $m_0 \neq m_1$ כך ש-$h(m_0) = h(m_1)$

מלכודת

ההבדל בין השתיים האחרונות: ב-second pre-image ההודעה הראשונה **נתונה**, וב-collision התוקף בוחר את **שתיהן**. תשובה עם "$h(m_0) \neq h(m_1)$" או "קשה למצוא 'd digest שונה מ-d" היא תמיד מסיח שגוי.

פרדוקס יום ההולדת

מספיקים 23 אנשים בחדר כדי שההסתברות ששניים חולקים יום הולדת תעלה על 50%. באופן כללי, מתוך N ערכים אפשריים, כ-$\sqrt{N}$ ניסיונות (בערך $1.2 \cdot \sqrt{N}$) נותנים סיכוי של 50% להתנגשות. לכן מציאת התנגשות ב-hash של n ביט עולה רק כ-$2^{n/2}$. זה עיקרון כללי, לא ייחודי להתקפת קמינסקי (משמש גם שם, פרק 9).

שמירת סיסמאות

לא שומרים סיסמה כמו שהיא. שומרים $h(password)$. בעיה: למשתמשים עם אותה סיסמה יש אותו hash, ותוקף שגנב את הטבלה בונה מראש מילון של hash-ים לסיסמאות נפוצות (brute force / מילון). לכן מוסיפים:

- **Salt**: מחרוזת אקראית וארוכה לכל משתמש, נשמרת **גלויה** בטבלה. שומרים $username,\ salt,\ h(password \parallel salt)$. שני משתמשים עם אותה סיסמה מקבלים hash שונה, ומילון מוכן מראש כבר לא עובד: צריך brute force נפרד לכל משתמש.
- **Pepper**: מחרוזת סודית שלא נשמרת ב-DB (למשל בקובץ סביבה בשרת). $h(password \parallel salt \parallel pepper)$. תוקף שגנב רק את ה-DB לא יודע אותה. גרסה נוספת: pepper קצר לכל משתמש שלא נשמר בכלל, והשרת מנסה את כל הערכים בכניסה.
- **Key stretching**: hash חוזר i פעמים, כל פעם על הפלט הקודם): מאט כל ניחוש של התוקף פי i. זו התוספת הכי חזקה מבין האפשרויות שבמבחן.

במבחן

- תוקף גנב קובץ hash-ים של סיסמאות: הוא יבצע **brute force** (מילון).
- $Username,\ hash(salt),\ hash(password \parallel salt)$: **לא פרקטי**, כי ה-salt עצמו לא נשמר והשרת לא יכול לאמת.
- $username,\ salt,\ h(password) \parallel h(salt \parallel pepper)$: **שימוש שגוי**. החלק של הסיסמה נשאר $h(password)$ בלי salt, והחלק השני קבוע. מילון עובד.
- משתמש אחר נכנס עם סיסמה שונה משל אליס: **hash collision**.
- לקוח ששולח $Hash(Username \parallel Password)$ על TCP רגיל: פגיע ל**מאזין**, שפשוט ישדר את אותו ערך שוב (replay). ה-hash הופך בפועל לסיסמה.

MAC (Message Authentication Code)

המקבילה הסימטרית לחתימה. לאליס ולבוב מפתח משותף k. אליס שולחת m ו-$tag = MAC_k(m)$; בוב מחשב בעצמו ומשווה. מבטיח **אמינות** (integrity, ההודעה לא שונתה) ו**אותנטיות** (נשלחה ע"י מי שמחזיק את k). **לא** מבטיח **אי-הכחשה** (non-repudiation): גם בוב יודע את k ויכול היה לייצר את ה-tag בעצמו.

שילוב הצפנה ו-MAC

שיטה	מה נשלח	מה המקבל עושה, ובאיזה סדר
Encrypt-and-MAC	`Enc(m), MAC(m)`	מפענח, ואז מחשב MAC על ה-**plaintext** ומשווה ל-tag שהגיע **גלוי** ליד ה-ciphertext
MAC-then-Encrypt	`Enc(m || MAC(m))`	קודם **מפענח**, מפריד בין ההודעה ל-tag (ה-tag היה **בתוך** ההצפנה), ואז בודק MAC על ה-plaintext
Encrypt-then-MAC	`c = Enc(m), MAC(c)`	קודם **בודק MAC על ה-ciphertext**, ורק אם תקין מפענח. השיטה המומלצת

במבחן: איך מזהים את השיטה מתיאור

שואלים שתי שאלות: (1) על מה חושב ה-MAC, על ה-plaintext או על ה-(2) ?ciphertext האם ה-tag מוצפן יחד עם ההודעה? MAC על ה-ciphertext הוא תמיד Encrypt-then-MAC. MAC על ה-plaintext כשה-tag מוצפן בפנים הוא MAC-then-Encrypt. MAC ה-plaintext כשה-tag נשלח גלוי הוא Encrypt-and-MAC.

מלכודת

אליס שולחת לבוב ולקרול $M,\ MAC_K(M),\ Enc_{PK_B}(K),\ Sign_{SK_A}(hash(K))$. החתימה היא רק על K, לא על ההודעה. בוב שקיבל את K יכול להצפין את אותו K לקרול, לצרף את אותה חתימה, ולכתוב MAC על הודעה שאליס לא שלחה. כלומר **אין אותנטיות**.

4 קריפטוגרפיה אסימטרית: דיפי-הלמן, RSA וחתימות

בהצפנה אסימטרית לכל צד זוג מפתחות: **פומבי** (pk), שכולם יודעים, ו**פרטי** (sk), שרק הוא יודע. כך פותרים את בעיית העברת המפתח, אבל החישובים כבדים ואיטיים בהרבה מהצפנה סימטרית.

רקע מתמטי: חבורות ולוגריתם דיסקרטי

- $Z_p^* = \{1, \ldots, p-1\}$ עבור p ראשוני, עם כפל מודולו p, היא **חבורה**: סגירות, אסוציאטיביות, איבר יחידה, ולכל איבר הופכי (למשל $3 \cdot 2 \bmod 5 = 1$, ולכן $3^{-1} \equiv 2$).
- **חבורה ציקלית**: יש **יוצר** g שהחזקות שלו עוברות על כל איברי החבורה.
- **בעיית הלוגריתם הדיסקרטי**: בהינתן p, g, a קשה למצוא x כך ש-$g^x \bmod p = a$. בכיוון ההפוך (חזקה) החישוב קל. על זה מתבססת דיפי-הלמן.

החלפת מפתחות דיפי-הלמן

1. מסכימים בגלוי על n (ראשוני) ו-g.
2. אליס בוחרת בסוד a ושולחת $A = g^a \bmod n$. בוב בוחר b ושולח $B = g^b \bmod n$.
3. אליס מחשבת $B^a \bmod n$, בוב מחשב $A^b \bmod n$. שניהם מקבלים $K = g^{ab} \bmod n$.

מאזין רואה n, g, A, B אבל לא יכול לחלץ את a או b בגלל בעיית הלוגריתם הדיסקרטי. אנלוגיית הצבעים מההרצאה: צבע ציבורי צהוב, כל צד מערבב צבע סודי, שולח את התערובת ומוסיף שוב את הסוד שלו.

מלכודת

הסדר בחזקה: מעלים את **g בחזקת x** ($g^x \bmod n$), ולא את x בחזקת g. זו התשובה השגויה הקבועה בשאלות "בחרו את הסעיף שאינו נכון".

חולשה

דיפי-הלמן בגרסה הבסיסית **פגיע ל-MitM**: התוקף מבצע החלפה נפרדת עם כל צד ומחזיק שני מפתחות. הפתרון: לאמת את ערכי ההחלפה, למשל לחתום עליהם או להצפין אותם במפתח הפומבי של הצד השני, ובפועל להשתמש בתעודות (פרק 5). מול **מאזין בלבד** הפרוטוקול בטוח.

במבחן: "איזו שיטה לא תעבוד מול מאזין?"

- × כל צד שולח בערוץ מפתח סימטרי שבחר: המאזין רואה אותו.
- × כל צד שולח את המפתח הפומבי שלו, ואז **מצפין במפתח הפומבי של עצמו**: רק הוא יכול לפענח. צריך להצפין במפתח הפומבי של **הצד השני**.
- ✓ דיפי-הלמן, תעודות של CA ודיפי-הלמן, או החלפת מפתחות פומביים והצפנה במפתח של הצד השני. כל אלה עובדים מול מאזין.

Textbook RSA

יצירת מפתחות (Gen): בוחרים ראשוניים p, q. מחשבים $n = p \cdot q$ ו-$\varphi(n) = (p-1)(q-1)$, כך ש-$e \cdot d \equiv 1 \pmod{\varphi(n)}$ זר ל-$\varphi(n)$. בוחרים e ו-d.

מפתח פומבי (e, n) · **מפתח פרטי** (d, n)

הצפנה $c = m^e \bmod n$ · **פענוח** $m = c^d \bmod n$

חתימה $\sigma = m^d \bmod n$ · **אימות** $\sigma^e \bmod n \stackrel{?}{=} m$

הנכונות: $(m^e)^d = m^{ed} \equiv m \pmod n$ כי $ed \equiv 1 \pmod{\varphi(n)}$. לכן אפשר גם להצפין בפרטי ולפענח בפומבי, וזה הבסיס לחתימה.

הצפנה היברידית

כדי ליהנות משני העולמות: בוחרים מפתח **סימטרי** אקראי, מעבירים אותו מוצפן ב**מפתח הפומבי** של הצד השני, ומשם ממשיכים בהצפנה סימטרית מהירה. כך בדיוק עובד TLS עם RSA (פרק 6).

חתימה דיגיטלית

רק בעל המפתח הפרטי יכול לחתום, וכל מי שיש לו את המפתח הפומבי יכול לאמת. החתימה נותנת **אמינות**, **אותנטיות**, ובניגוד ל-MAC גם **אי-הכחשה**, כי המפתח הפרטי שייך לאדם אחד.

החתימה הנכונה (כך מנוסחת התשובה הנכונה)

מחשבים $h(m)$, "מצפינים" אותו עם **המפתח הפרטי של השולח** ושולחים יחד עם ההודעה. המקבל "מפענח" עם **המפתח הפומבי של השולח** ומשווה ל-hash שהוא מחשב על ההודעה. כל וריאציה עם "פומבי של השולח לחתימה" או "פרטי של השולח לאימות" שגויה.

למה חותמים על $h(m)$ ולא על m? ב-textbook RSA החתימה כפלית: $(m_1 m_2)^d = \sigma_1 \cdot \sigma_2$, ולכן תוקף יכול לייצר חתימה חוקית על $m_1 \cdot m_2$ בלי המפתח. עם hash זה לא עובד, כי $h(m_1 m_2) \neq h(m_1) \cdot h(m_2)$. בנוסף, hash מקצר הודעות ארוכות.

במבחן: סודיות + אותנטיות + אמינות

אליס שולחת $E_{PKB}(M),\ S_{SKA}(H(M))$: הצפנה ב**פומבי של בוב** (רק בוב יפענח) וחתימה ב**פרטי של אליס** (רק אליס יכולה לחתום).
$E_{PKB}(s),\ Enc_s(M)$ נותן סודיות בלבד, בלי אימות שולח. הצפנה ב-PKA או חתימה ב-SKB שגויים מהיסוד.

במבחן: עוד ניסוחים שחוזרים

- אי-הכחשה (X לא יוכל להתכחש): **חתימות דיגיטליות** / public key cryptography. לא MAC.
- הצפנה נועדה ל**סודיות**; חתימה נועדה לאימות **מקור ושלמות**.
- שרת מפיץ מפתחות פומביים עם חתימה על (מפתח + תאריך ושעה): פגיע ל-MitM. החתימה לא כוללת **את שם בעל המפתח**, ולכן תוקף מבקש מהשרת את המפתח של עצמו, מקבל תשובה חתומה ועדכנית ומעביר אותה לאליס כאילו זה המפתח של בוב.

5 PKI ותעודות דיגיטליות

ההצפנה האסימטרית לא עונה על השאלה "האם המפתח הפומבי הזה באמת של google.com?". **PKI** (Public Key Infrastructure) היא התשתית שקושרת מפתח פומבי לזהות, באמצעות **תעודות** שחותם עליהן **CA** (Certificate Authority).

הנפקת תעודה

1. האתר מייצר בעצמו זוג מפתחות pk, sk. **המפתח הפרטי לא יוצא מהשרת ולא מופיע בתעודה.**
2. האתר פונה ל-CA. ה-CA מוודא בעלות על הדומיין (verify eligibility), למשל בקשה להוסיף רשומת DNS עם טקסט שה-CA נתן.
3. ה-CA חותם עם **המפתח הפרטי שלו** על התעודה, שמכילה: subject (הדומיין), pk של האתר, issuer (מי הנפיק), תוקף (from–to), חתימת ה-CA וextensions.

שרשרת אמון (Chain of Trust)

הדפדפן מאמת את חתימת ה-CA עם המפתח הפומבי של ה-CA, שמופיע בתעודה שחתם עליה CA אחר, וכן הלאה עד ל-**Root CA** שנמצא ב-**Root Store** של מערכת ההפעלה או הדפדפן. תעודת ה-Root חתומה על ידי עצמה (self-signed). כדי להגן על המפתח של ה-Root, הוא חותם רק על **Intermediate CA**, ואלה חותמים על תעודות האתרים.

במבחן

- ב-TLS **משתמשים** ב-chain of trust, והיא מה שמאפשר ללקוח לוודא את התעודה של foo.com. הלקוח בודק **חתימות תקינות לאורך השרשרת עד Root שהוא סומך עליו**.
- ה-CA שהנפיק לשרת את התעודה **לא חייב** להיות ב-Root Store (מספיק שיש שרשרת). Root CA חותם על המפתח הפומבי של intermediate CA, ולא ישירות על כל שרת.
- התעודה של foo.com חתומה ע"י ה-CA, לא ע"י foo.com. שרת אינטרנט **לא** מייצר תעודה לכל חיבור. אסור שיופיע בתעודה: **מפתח פרטי**.
- CA שהנפיק תעודה לאתר **לא** יכול לפענח את התעבורה שלו, כי אין לו את המפתח הפרטי של האתר.

ביטול תעודות (Revocation)

אם מפתח פרטי נגנב, צריך לבטל את התעודה לפני שפג תוקפה. מקרה מפורסם: **DigiNotar** (2011). ה-CA נפרץ, הונפקו תעודות מזויפות, הדפדפנים הוציאו אותו מרשימות האמון והחברה נסגרה. ביטול Root CA קשה במיוחד, כי צריך לעדכן מערכות הפעלה.

מנגנון	איך עובד	חולשות
CRL	רשימת מספרים סידוריים של תעודות שבוטלו, שה-CA מפרסם בכתובת שמופיעה בתעודה (CRL Distribution Points)	רשימות גדולות, ומתעדכנות מדי פעם (למשל פעם בשבוע), כך שיש חלון זמן
OCSP	הדפדפן שואל את שרת ה-OCSP של ה-CA בזמן אמת "האם התעודה תקפה?" ומקבל תשובה **חתומה ע"י ה-CA**	סבב תקשורת נוסף (extra round-trip), עומס, **בעיית פרטיות** (ה-CA יודע לאן גולשים), ו-**soft fail**
OCSP Stapling	**השרת** מביא מראש מה-CA תשובת OCSP חתומה ועדכנית, ו"מצמיד" אותה ל-handshake. הלקוח לא פונה ל-CA	פותר פרטיות, השהיה ועומס. בלי Must-Staple תוקף יכול להציג תעודה בלי staple
CRLSets	רשימת ביטולים מצומצמת שהדפדפן (Chrome) מוריד ומעדכן בעצמו	מכסה רק חלק מהתעודות

Soft fail: רוב הדפדפנים, אם לא הצליחו לקבל תשובת ביטול (שרת לא זמין, תקלה רגעית), **מקבלים את התעודה כתקפה** כדי לא לפגוע בחוויית המשתמש. תוקף MitM שחוסם את בקשת ה-OCSP גורם לדפדפן לקבל תעודה מבוטלת. לכן soft fail **לא** מסייע לעצור תוקף, ומסביר למה תעודה גנובה עדיין מתקבלת שעות אחרי הביטול.

מלכודות

- "OCSP stapling יעיל בזכות עץ מרקל": **שגוי**. עץ מרקל שייך ל-CT ולביטקוין.
- "OCSP משמש miners לבדיקת בלוק": **שגוי**. אין קשר בין OCSP לביטקוין.
- "OCSP stapling הוא מנגנון שונה לחלוטין מ-OCSP": התשובה זהה (אותה תשובה חתומה), רק מי שמביא אותה שונה.

מפתח פרטי של שרת נגנב: מה התוקף יכול

✓ יכול	× לא יכול
להתחזות לשרת מול לקוחות, אם הוא MitM; לחתום על מידע בשם השרת; לפענח תעבורה **ישנה** שהקליט, אם החיבור השתמש בהחלפת מפתחות RSA (אין forward secrecy)	לחתום על CRL או לזייף תשובות OCSP בשם ה-CA, כי אלה חתומים במפתח **של ה-CA**. האתר מבקש מה-CA לבטל, וזו טענה נכונה

Certificate Transparency (CT)

מה קורה אם CA עצמו נפרץ ומנפיק תעודה "בשקט" ל-google.com? CT (פותח ע"י Google) מחייב שכל תעודה תירשם ב**יומנים ציבוריים**. המטרה היא **גילוי** מהיר, לא מניעה.

- **Logger**: מתחזק יומן ציבורי, append-only, של תעודות שהונפקו, מבוסס **עץ מרקל** (אפשר להוסיף, אי אפשר למחוק).
- **SCT** (Signed Certificate Timestamp): הלוגר חותם על (התעודה + זמן). זו ההוכחה שה-CA דיווח. ה-SCT מוטמע בתעודה.
- **אכיפה בדפדפן**: הדפדפן דורש מספר SCT-ים מ**לוגרים שהוא סומך עליהם**. בקורס: Chrome דורש לוגר של Google ולוגר שאינו של Google. כיום (מאז 2022) Chrome דורש 2 SCT-ים לתעודה של עד 180 יום ו-3 לתעודה ארוכה יותר, משני מפעילים שונים לפחות.
- **Monitor**: מנטר את **הלוגרים** ושואל אותם על תעודות חדשות (למשל שירות של Facebook). כך בעל דומיין מגלה תעודה שלא ביקש. הלוגרים מחויבים לפרסם תעודה תוך MMD (Maximum Merge Delay).
- **Split-world**: לוגר שנשלט ע"י תוקף מציג עץ אחד לדפדפן ועץ אחר למוניטורים. עץ מרקל ובדיקות עקביות (gossip) נועדו לחשוף זאת. אותו רעיון שקיפות משמש את WhatsApp לניטור החלפות מפתחות.

מלכודות

- המוניטורים מנטרים את **הלוגרים**; הלוגרים מתחזקים יומן של תעודות שהנפיקו CA-ים. לא "המוניטורים מנטרים CA-ים".
- ה-CA חייב לרשום **אצל Logger**, לא אצל Monitor.
- "לא כל רשומה ב-log חייבת להיות גלויה לציבור": **שגוי**. כל היומן ציבורי.
- "SCT נועד לפתור בעיית פרטיות של OCSP": **שגוי**. את זה פותר OCSP stapling. CT ו-CRL/OCSP פותרים בעיות **שונות** (גילוי הנפקה זדונית לעומת ביטול).
- "ב-CT משתמשים ב-cookie למציאת התעודה הבאה": **שגוי**. ב-CT משתמשים בעץ מרקל.

6 תקשורת מאובטחת: TLS ו-HTTPS

TLS (לשעבר SSL) מצפין את החיבור בין דפדפן לשרת אחרי הקמת חיבור TCP. HTTPS = HTTP over TLS. בקורס לומדים את גרסת **RSA key exchange**: הצפנה היברידית, שבה RSA מעביר סוד, וממנו נגזרים מפתחות סימטריים.

ה-Handshake המלא (כפי שנלמד בקורס)

1. **ClientHello**: הלקוח שולח גרסה, רשימת cipher suites שהוא תומך בהם ומספר אקראי c_{rnd}.
2. **ServerHello**: השרת שולח s_{rnd}, את ה-cipher suite שבחר, session id ואת **התעודה** (subject + מפתח פומבי).
3. הלקוח **מאמת את התעודה** (שרשרת, תוקף, ביטול, ו-**שה-subject הוא השרת שאליו רצה להתחבר**). מגריל pre-master secret k_{ms} ושולח אותו מוצפן ב**מפתח הפומבי של השרת**: **ClientKeyExchange**. זו הפעולה הכבדה ביותר.
4. השרת מפענח עם המפתח הפרטי. שני הצדדים גוזרים מ-$(k_{ms}, c_{rnd}, s_{rnd})$ מפתחות **נפרדים לכל כיוון** ולכל תפקיד (הצפנה / MAC).
5. **ClientFinished**: הלקוח שולח, מוצפן ועם MAC, hash של **כל הודעות ה-handshake** עד כה. השרת מוודא עם **אותם מפתחות של הלקוח**.
6. **ServerFinished**: אותו דבר בכיוון השני. רק עכשיו ה-handshake הסתיים ומתחילה העברת המידע.

למה כל רכיב קיים

- **מספרים אקראיים** c_{rnd}, s_{rnd}: מבטיחים ש**המפתח הסימטרי יהיה שונה בכל חיבור** (גם ב-session resumption), ומונעים replay של handshake ישן.
- **Finished עם hash של כל ההודעות**: מגן מפני **התקפת שנמוך** (downgrade). MitM שמחק cipher suites חזקים מה-ClientHello ייתפס, כי ה-hash שהלקוח חישב לא יתאים למה שהשרת ראה. הוא **יתגלה מיד כשה-ClientFinished מגיע לשרת**.
- **מפתחות שונים לכל כיוון**: מונעים **התקפת reflection**, שבה התוקף מחזיר ללקוח הודעה שהלקוח עצמו שלח כאילו השרת שלח אותה.
- **Session resumption**: הלקוח שולח session id קודם. אם השרת זוכר אותו, הוא מחזיר **אותו** id, מדלגים על ClientKeyExchange, ונגזרים מפתחות חדשים מ-randoms חדשים. אחרת השרת מחזיר id אחר ומתבצע handshake מלא.

במבחן: מתי זה יתגלה?

- MitM שינה תו ב-ClientHello: **מיד אחרי שה-ClientFinished מגיע לשרת**.
- MitM החליף את תעודת השרת בתעודה **חוקית של דומיין אחר** (עם שרשרת תקינה): **מיד אחרי שה-ServerHello מגיע ללקוח**, כי ה-subject לא מתאים.
- "ב-ClientHello השרת מעביר ללקוח את ה-cipher suite": שגוי. ClientHello נשלח **מהלקוח**.
- "אחרי ClientKeyExchange ה-handshake מסתיים": שגוי. צריך עוד את הודעות ה-Finished.
- אחרי ה-handshake ההודעות מוצפנות **סימטרית**, לא ב-public key של השרת. TLS משתמש בהצפנה סימטרית **וגם** בא-סימטרית.
- אם החומרה תבצע קריפטו "באפס זמן", HTTPS עדיין איטי מ-HTTP, כי ה-handshake מוסיף **סבבי תקשורת**.
- אפשר להקים TLS בלי דיפי-הלמן (RSA key exchange), והשרת **לא חייב** לאמת תעודת לקוח.

מעבר לקורס, בקצרה

ב-TLS 1.3 (RFC 8446) הוסר ה-RSA key exchange הסטטי, וחובה להשתמש ב-(EC)DHE. כך מתקבלת **forward secrecy**: גניבת המפתח הפרטי לא חושפת תעבורה מוקלטת מהעבר. ה-handshake מתקצר לסבב אחד (1-RTT).

HSTS (HTTP Strict Transport Security)

הבעיה: משתמש מקליד bank.com, הדפדפן פונה ב-**HTTP** והשרת מפנה (redirect) ל-HTTPS. תוקף MitM מבצע **SSL stripping**: מול השרת הוא מדבר HTTPS, ומול הלקוח HTTP גלוי, וקורא הכל.

הפתרון: השרת שולח header `Strict-Transport-Security: max-age=…`. מאותו רגע הדפדפן פונה לדומיין **רק ב-HTTPS**, גם אם המשתמש מקליד `http://`. החולשה: **TOFU** (Trust On First Use), כי הביקור הראשון עדיין חשוף.

HSTS Preload: הדומיין נרשם מראש ברשימה (hstspreload.org) שמובנית בדפדפנים, כך שגם דפדפן **שמעולם לא גלש לאתר** יפנה אליו רק ב-HTTPS.

7 ביטקוין ובלוקצ'יין

ביטקוין (סאטושי נקמוטו, 2008) הוא מטבע **מבוזר**: אין בנק שמנהל את הספר, ואין מי שיקפיא חשבון או יבטל עסקה. הבעיה המרכזית של כסף דיגיטלי היא **double spending**, כלומר שימוש באותו מטבע פעמיים. ביטקוין פותר אותה בלי צד שלישי. 1 סאטושי = 10^{-8} ביטקוין.

מפתחות, ארנקים ועסקאות

- **ארנק** = זוג מפתחות **א-סימטריים** (ECDSA). הכתובת נגזרת מ-hash של המפתח הפומבי; כדי לבזבז חותמים עם המפתח הפרטי. אין שימוש ב-OTP בארנקים. ארנק חם מחובר לרשת, ארנק קר לא.
- **עסקה** מבזבזת פלטים של עסקאות קודמות ויוצרת פלטים חדשים. מאחר שכל ההיסטוריה ציבורית, כל מי שמחזיק את הבלוקצ'יין (למשל miner) יכול לחשב כמה ביטקוין יש לכל כתובת. הבעלים של הכתובות לא מזוהים, אבל העסקאות גלויות.
- כדי לבצע עסקאות **לא** צריך לכרות. מי שמאמת עסקאות ומכניס אותן לבלוק הם **הכורים**.

הבלוקצ'יין ועץ מרקל

הספר (ledger) הוא שרשרת בלוקים. כל בלוק מכיל **prev hash** (hash של הבלוק הקודם), **שורש עץ מרקל** של העסקאות שלו, nonce וזמן. שינוי של בלוק ישן משנה את ה-hash שלו ושובר את כל השרשרת אחריו.

עץ מרקל: עץ בינארי שהעלים שלו הם hash-ים של עסקאות, וכל צומת הוא $h(\text{שמאל} \parallel \text{ימין})$, עד השורש. **הוכחת הכללה** (proof of inclusion): כדי להוכיח שעסקה בבלוק, מספיק לתת את ה-hash-ים של ה"אחים" לאורך המסלול לשורש, כלומר $\log_2(n)$ ערכים. לקוח קל (SPV) מאמת כך עסקה בלי להחזיק את כל העסקאות. עץ מרקל **לא** מספק פרטיות ולא הצפנה.

> **במבחן**
> 1024 עסקאות בבלוק: לקוח קל צריך $\log_2 1024 = 10$ hash-ים. שינוי עלה אחד בעץ: משתנים **רק הצמתים במסלול מהעלה עד השורש (כולל)**.

כרייה: Proof of Work

- כדי להוסיף בלוק, הכורה מחפש **nonce** שגורם ל-hash של כותרת הבלוק להתחיל במספיק אפסים (להיות קטן מ-target). הדרך היחידה היא ניסוי וטעייה. אימות הפתרון, לעומת זאת, דורש hash אחד.
- הקושי מתעדכן כל **2016 בלוקים** כדי לשמור על בלוק כל **~10 דקות**. הפרס נחצה כל **210,000 בלוקים** (כ-4 שנים). הכורה מקבל פרס בעסקה הראשונה בבלוק (coinbase) וגם את העמלות.
- **למה 10 דקות?** לוקח זמן עד שבלוק מתפשט ברשת (flooding via gossiping). אם בלוקים היו נמצאים כל 10 שניות כשההפצה לוקחת דקה, היו נוצרים המון fork-ים והכורים היו מבזבזים עבודה על שרשראות שיימחקו.
- **Mining pools**: היום כורים בעיקר בקבוצות שמחלקות ביניהן את מרחב החיפוש ואת הפרס. כורה בודד בבית ככל הנראה יפסיד כסף.

בדיקות שמחשב מבצע לפני שהוא מוסיף בלוק: ה-hash מתחיל במספיק אפסים; כל העסקאות חתומות כחוק; אין עסקאות סותרות בתוך הבלוק או מול השרשרת (double spending); ה-prev hash מצביע לסוף השרשרת. המחשב **לא** מוסיף חתימה משלו לבלוק.

קונצנזוס, Forks ו-Double Spending

- **כלל השרשרת הארוכה** (ליתר דיוק, זו עם הכי הרבה עבודה): כששני בלוקים נמצאים במקביל נוצר fork, וכל צומת ממשיך על השרשרת הארוכה ביותר שהוא מכיר. הענף הקצר ננטש.
- **מניעת double spending**: מחכים מספיק בלוקים (בדרך כלל 6) מעל העסקה (depth). כדי לשכתב היסטוריה, התוקף צריך לבנות שרשרת ארוכה יותר מכל הרשת, ואין לו מספיק כוח חישוב.
- אליס ששינתה בלוק בעותק המקומי שלה לא תצליח, **כל עוד השרשרת שלה לא ארוכה מהמקורית**. אין "בנקאי" שמאשר.
- **51%**: מי ששולט ברוב כוח החישוב יכול לקבוע אילו עסקאות ייכנסו, למנוע מעסקאות של משתמש מסוים להיכנס (exclusion) ולבצע double spending. הוא **לא** יכול לגנוב מטבעות של אחרים, כי אין לו את המפתחות שלהם.
- **Sybil attack**: יצירת המון זהויות מזויפות. PoW לא מושפע מזה, כי ההשפעה תלויה בכוח חישוב ולא במספר זהויות.

• לקוח "רזה" ששומר רק את העסקאות שלא בוזבזו ואת הבלוק האחרון **לא יכול להתמודד בעצמו עם fork-ים**, כי אין לו את ההיסטוריה כדי לחזור אחורה.

> **מלכודת**
> הצעה "לכרות רק כשיש x עסקאות חדשות כדי לחסוך חשמל, וזה לא מאפשר double spending": הטענה **שגויה**. בזמן שהכורים הישרים מחכים, תוקף ממשיך לכרות שרשרת סודית וצובר יתרון.

Soft fork לעומת Hard fork

Soft fork	Hard fork
הכללים **מוחמרים**: בלוקים חדשים עדיין חוקיים בעיני צמתים ישנים. דוגמה: הקטנת בלוק מ-2MB ל-1MB	הכללים **מורחבים או משתנים**: צמתים ישנים **דוחים** בלוקים חדשים, והרשת מתפצלת לשני מטבעות. דוגמאות: הגדלת בלוק (Bitcoin Cash, 2017), **החלפת פונקציית ה-hash של ה-PoW**

איך מוכיחים בעלות על מטבע (P2PKH)

פלט של עסקה "ננעל" ל-**hash של מפתח פומבי** (זו הכתובת). כדי לבזבז אותו, המבזבז חייב להציג שני דברים: **את המפתח הפומבי עצמו**, שה-hash שלו שווה לכתובת, ו**חתימה דיגיטלית על העסקה במפתח הפרטי** המתאים. הכורים בודקים קודם שה-hash של המפתח מתאים לכתובת, ואחר כך שהחתימה תקינה מול המפתח.

> **במבחן**
> מה המשתמש חייב לספק כדי לבזבז? **מפתח פומבי + חתימה במפתח הפרטי**. לא hash של המפתח (הוא כבר רשום בפלט), ולא "חתימה במפתח הפומבי" (אי אפשר לחתום בפומבי). בשאלה שמציגה סדרת פעולות, הסדר התקין: חתימה, מפתח פומבי, שכפול המפתח, חישוב hash שלו, השוואה ל-hash שבפלט, בדיקת החתימה.

עסקאות מתקדמות

• **Escrow / Multisig**: ארנק משותף לקונה, מוכר וצד שלישי מוסכם, שדורש **2 מתוך 3** חתימות. אם הכל תקין, קונה ומוכר חותמים **בלי** המתווך. בסכסוך, המתווך מכריע עם אחד הצדדים. לכן "המתווך חייב לאשר" ו"המוכר חייב לאשר" שגויים.

• **Micropayments**: אליס נועלת כסף בארנק משותף עם בוב, ומראש מקבלת עסקת החזר עם **LockTime** (עסקה שלא תיכנס לבלוק לפני זמן מסוים). כל תשלום קטן הוא עסקה חדשה שאליס חותמת. בוב מפרסם רק את האחרונה. אליס יודעת שבוב ישחרר את מה שמגיע לה, **כי אחרת, כשה-LockTime יגיע, כל הכסף יחזור אליה ובוב יפסיד**.

• **Proof of Stake**: הסיכוי לבחור את הבלוק הבא תלוי בכמות המטבעות שמחזיקים, לא בעבודה חישובית. חוסך חשמל.

• **Ethereum**: פלטפורמה לחוזים חכמים. **Namecoin**: בלוקצ'יין לניהול שמות דומיין במקום DNS. **BIP**: מסמך הצעת שיפור לביטקוין.

8 אבטחת Web

SQL Injection

השרת בונה שאילתה למסד הנתונים על ידי **שרשור טקסט**: קטע קבוע, ואחריו מה שהמשתמש הקליד, בתוך גרשיים. אם הקלט עצמו מכיל **גרש**, הגרש סוגר את המחרוזת מוקדם, ומה שבא אחריו כבר לא נקרא כנתון אלא **כחלק מהלוגיקה של השאילתה**. כך התוקף משנה את מבנה השאילתה.

- **עקיפת התחברות:** סיסמה שמסתיימת בגרש ואחריו "או 1 שווה 1" הופכת את תנאי הבדיקה ל"שם משתמש וסיסמה תואמים, **או** `1=1`". מאחר ש-AND מחושב לפני OR, התנאי תמיד אמת. דרך אחרת: גרש ואחריו סימן הערה (`--`), שמבטל את המשך השאילתה, כולל בדיקת הסיסמה.
- **מה לא עובד:** קלט בלי גרש נשאר בתוך המחרוזת ולא משנה את המבנה. קלט שמחבר תנאי **AND** שתמיד אמת לא עוקף כלום, כי הסיסמה עדיין נבדקת.
- **עוד יכולות:** הוספת שאילתה שנייה (למשל מחיקת טבלה), שליפה מטבלה אחרת (UNION), ועקיפת סינון שמבוסס על **רשימה שחורה** (קידוד התווים, או מילה שבנויה כך שאחרי מחיקה אחת של מילה אסורה נשארת המילה האסורה).

Blind SQL injection: התוקף לא רואה את תוצאת השאילתה, אבל שואל שאלות כן/לא ומסיק את התשובה בעקיפין: האם הדף מוצג או לא, או כמה זמן לוקח לשרת לענות. כך הוא מגלה סיסמה תו אחר תו.

הגנות: שאילתות עם פרמטרים (prepared statements, stored procedures): מבנה השאילתה מוגדר מראש, והקלט תמיד נשאר נתון ולעולם לא קוד. בנוסף: ORM, ולידציה **בצד השרת** (חובה; בצד הלקוח רק נוחות), escaping וקידוד, ואי-הצגת הודעות שגיאה למשתמש. **IDS לא מונע injection.** הודעות כניסה צריכות להיות כלליות ("שם משתמש **או** סיסמה שגויים").

עוגיות ו-Same-Origin Policy

עוגייה: מחרוזת שהשרת קובע והדפדפן מצרף **אוטומטית** לכל בקשה לאותו אתר, למשל session token.

SOP: מנגנון **בדפדפן**. סקריפט מ-origin אחד לא יכול **לקרוא** מידע של origin אחר. **Origin = פרוטוקול + דומיין מלא + פורט.** הנתיב לא חלק מה-origin.

SOP חוסם	SOP לא חוסם
סקריפט מ-foo.com שקורא תוכן של תשובה מ-bar.com, או ניגש לתוכן של iframe מ-origin אחר	טעינת תמונה או סקריפט מדומיין אחר; שליחת טופס ל-bar.com (ולכן **לא מונע CSRF**); סקריפט שהוזרק ל-foo.com ושולח עוגיות ל-bar.com

> **מלכודת**
> `http://foo.com/~alice` ו-`http://foo.com/~bob` הם **אותו origin**. לכן iframe באתר של בוב **יכול** לקרוא את האתר של אליס, ו-SOP לא יחסום. הטענה שהוא "יגלה ש-SOP ימנע" שגויה.

XSS (Cross-Site Scripting)

התוקף גורם לקוד שלו לרוץ בדפדפן של הקורבן **כאילו הגיע מהאתר הפגיע**, וכך עוקף את SOP: גונב עוגיות או tokens, מציג תוכן מזויף ופועל בשם המשתמש. **הפרצה נמצאת בקוד האתר הפגיע**, לא בדפדפן.

סוג	מאיפה מגיע הקוד	איפה החולשה
Stored / Persistent	נשמר ב-DB (תגובה, פרופיל) ומוחזר לכל מי שגולש	שרת
Reflected	פרמטר בבקשה (קישור זדוני) שהשרת משקף לתוך הדף	שרת
DOM-based	קוד JavaScript של הדף קורא פרמטר (למשל מה-URL) בצורה לא בטוחה ומריץ אותו	**קוד שרץ בדפדפן** (צד לקוח)

ב-stored וב-reflected הקוד הזדוני חוזר כחלק מתשובת השרת. ניצול SQL injection כדי לשתול סקריפט ב-DB (ומשם XSS) הוא תרחיש הגיוני.

הגנות: חיטוי וקידוד פלט (sanitization, encoding; בצד השרת), עוגיות **HttpOnly** (JavaScript לא יכול לקרוא אותן, הן רק מצורפות לבקשות), ו-**CSP** (Content-Security-Policy: header שמגדיר לדפדפן מאילו origins מותר לטעון ולהריץ סקריפטים ומשאבים; כולל גם frame-ancestors).

> **במבחן**
> שם שמוצג כ-' במקום גרש: האתר משתמש ב**קידוד (encoding)** כדי למנוע injection. הקידוד הוצג פעמיים או במקום הלא נכון.

CSRF / XSRF (Cross-Site Request Forgery)

אתר זדוני גורם לדפדפן של הקורבן לשלוח בקשה לאתר שבו הקורבן מחובר. הדפדפן **מצרף אוטומטית את העוגיות**, והשרת **לא יודע** שהבקשה לא באמת של הקורבן. דוגמה קלאסית: שינוי מצב דרך **GET** (`/change-password?user=alice&password=1234`, או `foo.com/Bob?m=msg`). מספיק שהקורבן יבקר באתר של התוקף.

הגנות:

- **Anti-CSRF token**: ערך אקראי שהשרת מטמיע בטופס (שדה hidden) ובודק בכל בקשה. התוקף לא יכול לקרוא אותו בגלל SOP. **שקוף למשתמש**, שלא מקליד אותו ידנית.
- **cookies SameSite** (Strict/Lax): הדפדפן לא מצרף את העוגייה לבקשות שמקורן באתר אחר.
- בדיקת Referer/Origin header, custom headers, ו-CAPTCHA לפעולות רגישות.

> **מלכודות**
> - SOP **לא מונע** CSRF. ב-**stored XSS** צירוף העוגיות **לא** מהותי (זה מהותי ב-CSRF).
> - XSS **שובר** הגנת CSRF: הסקריפט רץ מתוך ה-origin, קורא את ה-token מהטופס ושולח בקשה תקינה. "ה-token לא יכול להיגנב כי הוא ב-POST" שגוי.
> - התקפת view-source ב-iframe, שבה המשתמש מתבקש להקליד טקסט מאזור ב-HTML: עוקפת **anti-CSRF token**.

Clickjacking

האתר הנתקף נטען ב-iframe שקוף מעל כפתור תמים, והמשתמש לוחץ בלי לדעת על כפתור באתר האמיתי. הגנה: `X-Frame-Options: SAMEORIGIN/DENY` או `CSP frame-ancestors`. X-Frame-Options **לא** נועד להגנה מ-SQL injection.

9 אבטחת רשת: ARP, DNS, DoS וחומות אש

הרעלת ARP (ARP poisoning / spoofing)

ARP ממפה כתובת IP לכתובת MAC ברשת המקומית: שואלים ב-broadcast "למי שייך 192.168.0.2?" ושומרים את התשובה בטבלת ARP (cache). שתי חולשות: **מקבל תשובות שלא ביקש**, ו**התשובה האחרונה דורסת** את הקודמת. אין שום אימות.

ההתקפה: התוקף (MAC ...:66:66) שולח לאליס "ה-IP של בוב נמצא ב-...:66:66", ולבוב את אותו הדבר על אליס. מעכשיו אליס שולחת חבילות ל-**IP של בוב** (192.168.0.2) אבל ל-**MAC של התוקף** (66:66:66:66:66:66). ה-switch מנתב לפי MAC בלבד, והתוקף מעביר הלאה. כך תוקף **off-path** ברשת המקומית הופך ל-**MitM**.

זיהוי והגנה: הרבה כתובות IP עם אותו MAC מעידות על התקפה; רשומות ARP **סטטיות** (מיפוי קבוע של IP ל-MAC); כלים על ה-switch (Dynamic ARP Inspection) ועל המחשב. מניעה **לא** אומרת להפסיק להשתמש בכתובות MAC.

DNS והרעלת DNS

DNS מתרגם שם (www.bob.com) לכתובת IP. הלקוח שואל resolver, שפונה ל-root, משם ל-TLD ומשם לשרת המוסמך, ושומר תשובות ב-cache לפי TTL. DNS רץ מעל **UDP** (פורט 53 בשרת), ולכן קל לזייף כתובת מקור. כל שאילתה מזוהה ב-**Transaction ID** של 16 ביט, והתשובה חייבת להגיע לפורט שממנו נשלחה השאלה. מבנה התשובה: Query, Answer, Authority, Additional.

המטרה: לשתול ב-cache של ה-resolver מיפוי שקרי (www.bob.com → 6.6.6.6), כך שכל לקוחותיו יגיעו לתוקף.

תוקף	מה צריך כדי להצליח
MitM (בין resolver ל-root)	לא צריך לנחש כלום: רואה את השאלה ועונה בעצמו (כשאין מידע ב-cache)
מאזין	רואה את ה-TXID והפורט, אבל לא יכול לחסום את התשובה האמיתית. **ב-DNS התשובה הראשונה קובעת** (עד שה-TTL פג), ולכן הוא צריך לנצח במרוץ, ו**לא תמיד יצליח**
Off-path	לנחש TXID (2^{16} : 1) ואת פורט היעד, ולענות לפני השרת האמיתי. לפי פרדוקס יום ההולדת, כ-$\sqrt{(2^{16})} = 256$ ניסיונות נותנים סיכוי של כ-50%

> **ARP מול DNS**
> ב-**ARP התשובה האחרונה דורסת**; ב-**DNS התשובה הראשונה קובעת** ולא נדרסת למשך ה-TTL. זו טענה שחוזרת שוב ושוב.

כלל Bailiwick: resolver מקבל רשומות רק לתחום שהשרת העונה מוסמך עליו. אם ns.evil.com מצרף בשדה Additional כתובת ל-google.com, הרשומה נזרקת.

התקפת קמינסקי: הבעיה של off-path היא שאחרי ניחוש שנכשל התשובה נשמרת ב-cache, וצריך לחכות TTL. קמינסקי שואל על תתי-דומיינים **שלא קיימים** (1.foo.com, 2.foo.com,...). אין להם cache, ולכן כל שאלה פותחת הזדמנות חדשה ללא הגבלה. בתשובה המזויפת התוקף מצרף ב-Authority/Additional את **השרת המוסמך של כל foo.com** עם ה-IP שלו (מותר לפי bailiwick), ומשתלט על הדומיין כולו. זו התקפת הרעלה, **לא התקפת DoS**.

הגנות: פורט מקור אקראי ($2^{32} = 2^{16} \cdot 2^{16}$ אפשרויות); **0x20 encoding** (אותיות גדולות וקטנות אקראיות בשם: wWw.GoOGle.cOm, והתשובה חייבת להחזיר בדיוק); צמצום חלון ההזדמנויות; **DNSSEC** (תשובות חתומות). **IDS לא מונע** הרעלת DNS, ו**IP spoofing** היא הטכניקה שבה ההתקפה משתמשת.

מניעת שירות: DoS ו-DDoS

המטרה: למנוע מהשרת לשרת לקוחות לגיטימיים. ב-**DDoS** ההתקפה מגיעה ממחשבים רבים (botnet), ולכן קשה להבדיל בין תוקף ללקוח וחסימת IP לא עוזרת. **כוח ההמונים** הוא הכוח המניע. DoS **לא** מוגבל לשכבות התחתונות, וגם לא ראשי תיבות של Disk on Stack.

משפחה	דוגמאות
ניצול באג (abuse)	Ping of Death, Land: חבילה אחת או בודדות שמקריסות את המערכת
התשת משאבים	SYN flood (זיכרון), HTTP/UDP flood, **Slowloris**: פותח הרבה חיבורי HTTP ומחזיק אותם פתוחים כמה שיותר זמן עם בקשות חלקיות (low-rate)
התשת רוחב פס	**Reflection**: בקשות עם IP מקור מזויף של הקורבן, והתשובות הולכות אליו. **Amplification**: התשובה גדולה מהבקשה (DNS amplification דרך open resolver)

SYN flood: התוקף שולח המון SYN ולא משלים את ה-handshake. השרת שומר מצב (TCB) לכל חיבור חצי-פתוח עד שהזיכרון נגמר. המטרה היא זיכרון השרת, לא רוחב הפס. התוקף **מזייף כתובות מקור, ובעיקר כאלה שלא עונות**: כך אף אחד לא שולח **RST** על ה-SYN+ACK (שהיה משחרר את המשאב), וחסימת כתובות ספציפיות לא עוזרת.

SYN cookies: תחת עומס השרת **לא שומר מצב**. הוא מקודד את פרטי החיבור במספר הסידורי (sequence number) של ה-SYN+ACK באמצעות hash עם סוד, ומשחזר אותם רק כשמגיע ACK תקין. כך עדיין אפשר לשרת לקוחות לגיטימיים.

> **במבחן**
> ב-**HTTP flood** מ-botnet החיבורים אמיתיים, ולכן **Source IP ו-Source Port שונים**, ו-**Destination IP ו-Destination Port זהים** (השרת, פורט 80/443).

Botnets, סריקות וכלים

- **Botnet**: Bot master שולט בשרתי **C&C** (Command & Control), והם שולטים בבוטים. כדי לנטרל, מפילים את ה-C&C. שימושים: DDoS, ספאם ופישינג, הונאת קליקים על פרסומות, קישורים מזויפים לקידום, כרייה. **Mirai**: botnet של מכשירי IoT שנפרצו בסיסמאות ברירת מחדל.
- **סריקות**: אנכית (כל הפורטים של כתובת אחת) או רוחבית (פורט מסוים בהרבה כתובות). **nmap** מגלה פורטים ושירותים פתוחים, ומסייע גם למגן לזהות חולשות. **Metasploit** הוא **כלי ניצול חולשות**, **לא** חומת אש. Hail Mary מנסה את כל ה-exploits בבת אחת. Kali Linux היא הפצה עם כלים אלה.
- **ערוץ צד של IP-ID**: אם מערכת ההפעלה של V מעלה מונה גלובלי ב-1 לכל חבילה, התוקף שולח ping בתחילת פרק זמן ובסופו. הפרש גדול מ-1 ב-IP-ID אומר ש-V שלח חבילות לאחרים בינתיים.

חומות אש (Firewalls)

סוג	איך מחליט	מגבלה
Packet filter, stateless	כל חבילה בנפרד, לפי IP, פורט, פרוטוקול וכיוון	לא רואה שכבת אפליקציה (תוקף מפצל "ipt"+"scr"), לא זוכר הקשר
Stateful	זוכר חיבורים: מאשר תשובה רק לחיבור שנפתח מבפנים	עדיין לא מבין תוכן
Application layer / WAF	בודק תוכן (HTTP), למשל לפי רשימת OWASP Top 10	כבד ואיטי; לכן ממקמים אותו אחרי packet filter

Whitelist: ברירת המחדל **drop**, ועוברת רק חבילה שתואמת חוק Allow. זהיר יותר. **Blacklist**: ברירת המחדל לעבור, ונחסם רק מה שתואם חוק חסימה.

> **במבחן**
> - חומת אש שלא עובדת בשכבת האפליקציה **כן** מגינה מפני **spoofing**: חבילה מבחוץ עם כתובת מקור פנימית נזרקת. היא **לא** מגינה מפני אימייל עם קוד זדוני, חולשה במימוש של החומה עצמה או תוקף שכבר בפנים.
> - חומה שחוסמת אוטומטית כתובות עם הרבה תעבורה **עלולה לחסום מחשבים לגיטימיים**. מחשב זדוני בפנים **יכול** לזייף כתובות מקור ולגרום לחסימת אחרים.

מלכודת חוקי ה-SSH

החוקים מאפשרים: (1) נכנס TCP לפורט 22 פנימי; (2) יוצא **מפורט מקור 22**; (3) יוצא לפורט 22 חיצוני; (4) נכנס **מפורט מקור 22** לכל פורט פנימי; (5) Drop. חוק 4 נועד לתשובות של שרתי SSH חיצוניים, אבל תוקף מבחוץ ששולח **מפורט מקור 22** לפורט 8080 עובר אותו ו**יכול** להקים חיבור לשרת הפרוקסי. הוספת עמודת **flags** (לאפשר בחוק 4 רק חבילות עם ACK) הייתה משפרת. הטבלה היא whitelist, ואכן מאפשרת SSH בשני הכיוונים.

- **DMZ**: תת-רשת נפרדת לשרתים שפונים אליהם מבחוץ (web, mail), מופרדת מהרשת הפנימית. נהוג להשתמש בשתי חומות אש, רצוי של יצרנים שונים. DMZ **לא** נועד למצב "שבו אי אפשר להשתמש בחומת אש".
- **IDS** (Intrusion Detection System): מערכת **פסיבית** שמזהה תבניות ואנומליות ו**מתריעה**. יכולה לעבוד offline על עותק של התעבורה, ולכן **לא מונעת**. **IPS**: מזהה **וגם** חוסם. **Honeypot**: מחשב שאף אחד לא אמור לפנות אליו, ולכן כל פנייה אליו חשודה. תוקפים מתחמקים בסריקות איטיות, obfuscation, פיצול חבילות ושינוי אותיות (`sCrIpT`).
- **SSH tunnel**: עוקף הגבלות גלישה דרך מחשב חיצוני שמשמש כ-proxy. **VPN**: חיבור מוצפן לשרת שגולש בשמך; ספק ה-VPN יודע לאן גלשת. **TOR**: ניתוב דרך 3 צמתים אקראיים עם הצפנה בשכבות ("בצל"); אף צומת לא רואה גם מקור וגם יעד. **Mix net**: מערבב הודעות ומשהה אותן כדי לשבור קשר תזמון. איטי, מתאים לדואר.

10 טענות שחוזרות במבחנים

אלה הטענות שמופיעות שוב ושוב, לפעמים כתשובה הנכונה ולפעמים כמסיח. מי שמזהה אותן מיד חוסך זמן ומונע טעויות.

✓ נכון

- MAC שמיושם נכון מספק אמינות מידע (integrity).
- בעיה גדולה ב-OTP היא העברת המפתחות.
- ב-TLS משתמשים בשרשרת אמון, וגם בהצפנה סימטרית וגם בא-סימטרית.
- ב-TLS משתמשים במפתחות שונים לכל כיוון (מונע reflection).
- CSP מגדיר מאילו כתובות מותר לטעון ולהריץ משאבים.
- SameSite=Strict מסייע נגד CSRF; SOP לא מונע CSRF.
- ב-DOM-based XSS הקוד הפגיע רץ בדפדפן.
- בביטקוין משתמשים במפתחות א-סימטריים, בעץ מרקל וב-Proof of Work.
- Root CA חותם על המפתח הפומבי של intermediate CA.
- CT מוסיף יומן ציבורי שבעזרתו מזהים הנפקה זדונית.
- OCSP stapling: השרת שולח סטטוס חתום ומעודכן, והלקוח לא פונה ל-CA.
- תוקף off-path מספיק כדי להרעיל ARP.
- הרעלת ARP מעניקה לתוקף יכולות MitM.
- ARP: האחרונה דורסת. DNS: הראשונה קובעת.
- פורט מקור אקראי ב-DNS מקשה על ניחוש.
- Slowloris מחזיק הרבה חיבורי HTTP פתוחים כמה שיותר זמן.
- Mining pools הן היום שיטת הכרייה הנפוצה.
- פרדוקס יום ההולדת הוא עיקרון כללי, לא ייחודי לקמינסקי.
- nmap יכול לסייע לזהות חולשות.

× לא נכון

- הגדרה נכונה של IDS מונעת הרעלת DNS.
- הרעלת DNS מחייבת יכולות MitM.
- Metasploit הוא חומת אש אפליקטיבית.
- x-frame-options מיועד להגנה מפני SQL injection.
- Anti-CSRF token דורש מהמשתמש להקליד קוד ידנית.
- ב-OTP הבטיחות תלויה בטקסט שמצפינים.
- בעיה גדולה ב-RSA / Diffie-Hellman / public key היא העברת המפתחות.
- OCSP stapling יעיל בזכות עץ מרקל.
- SCT נועד לפתור בעיית פרטיות של OCSP.
- CA חותם על מפתחות שה-CA יצר עבור השרת.
- שרת אינטרנט מייצר תעודה לכל חיבור TLS.
- הקמת TLS מחייבת תעודה רק בין שני שרתים.
- ב-escrow המתווך (או המוכר) חייב לאשר כל עסקה.
- צריך לכרות ביטקוין כדי לבצע עסקאות.
- בביטקוין משתמשים ב-OTP לייצור ארנקים.
- עץ מרקל מבטיח פרטיות של עסקאות.
- Syn cookies מכריחים את השרת לשמור TCB מה-SYN הראשון.
- אי אפשר למנוע התקפות שנמוך נגד switch.
- התקפת קמינסקי היא בעיקרה DoS.
- DoS נעשה רק בשכבות הפיזית, הלינק, התעבורה והרשת.
- מטרת SYN flood היא להעמיס על רוחב הפס.
- אפשר להשתמש בבוטים רק ל-DDoS.

אסטרטגיה לשאלת "איזו טענה אינה נכונה"

מחפשים את המילים המוחלטות ("תמיד", "אך ורק", "חייב", "לחלוטין") ואת ערבוב המושגים (OCSP עם miners, CT עם cookie, עץ מרקל עם OCSP). כמעט תמיד הן מסמנות את המסיח.

11 דף נוסחאות ודוגמאות פתורות

צפנים קלאסיים

הזזה (מספור $a = 0 \dots z = 25$):

$c = (m + k) \bmod 26 \; ; \; m = (c - k) \bmod 26$

קיסר: $k = 3$

Vigenère: $c_i = (m_i + k_{i \bmod |k|}) \bmod 26$

מרחב מפתחות (הזזה · החלפה · החלפה כשמיפוי אחד ידוע):

```
shift 25 · subst 26! · one pair known 25!
```

XOR ו-OTP

$c = m \oplus k \cdot m = c \oplus k$

$x \oplus x = 0 \cdot x \oplus 0 = x \cdot A \oplus B \oplus B = A$

Two-time pad: $c_1 \oplus c_2 = m_1 \oplus m_2$

Stream: $c = m \oplus \mathrm{PRG}(k)$

Modes of operation

ECB: $C_i = E_k(P_i)$

CBC: $C_i = E_k(C_{i-1} \oplus P_i),\ C_0 = IV$

3DES (EDE): $C = E_{K3}(D_{K2}(E_{K1}(m)))$

DES: בלוק 64 ביט, מפתח 56 · AES: בלוק 128

Hash וסיסמאות

חד-כיווניות: נתון d, קשה למצוא m כך ש-$h(m) = d$

Second pre-image: נתון m_0, קשה למצוא $m_1 \neq m_0$ עם hash זהה

Collision: קשה למצוא זוג כלשהו $m_0 \neq m_1$ עם hash זהה

שמירת סיסמה:

```
user, salt, h(pass || salt || pepper)
```

דיפי-הלמן

$A = g^a \bmod n \cdot B = g^b \bmod n$

$K = B^a = A^b = g^{ab} \bmod n$

בטיחות: לוגריתם דיסקרטי. פגיע ל-MitM

RSA

$n = p \cdot q \cdot \varphi(n) = (p-1)(q-1)$

$(d,n)\ \mathrm{sk} \cdot (e,n)\ \mathrm{pk} \cdot e \cdot d \equiv 1 \pmod{\varphi(n)}$

הצפנה $c = m^e \bmod n$ · פענוח $m = c^d \bmod n$

חתימה $\sigma = m^d \bmod n$ · אימות $\sigma^e \bmod n = m$

הסתברות וגדלים

יום הולדת: כ-$\sqrt{N}$ ניסיונות ל-50% התנגשות (23 אנשים)

TXID ב-DNS: 2^{16}; עם פורט אקראי: 2^{32}; $\sqrt{2^{16}} = 256$

הוכחת מרקל: $\log_2 n$ hash-ים (מעגלים כלפי מעלה)

ביטקוין: מספרים

בלוק כל ~10 דקות · קושי מתעדכן כל 2016 בלוקים

חציית פרס כל 210,000 בלוקים (~4 שנים)

המתנה של ~6 בלוקים · 1 סאטושי = 10^{-8} BTC

טריק החישוב: חזקה מודולרית בלי מחשבון

לא מחשבים 20^7 במלואו. מצמצמים מודולו n אחרי כל כפל, ומשתמשים בריבועים חוזרים. מספר שלילי קטן קל יותר: $20 \equiv -13 \pmod{33},\ 26 \equiv -7,\ 30 \equiv -3$.

דוגמה 1 · חתימת RSA

נתון $n = 33,\ d = 7,\ m = 5$. מה החתימה $\sigma = 5^7 \bmod 33$?

$5^2 = 25$

$5^4 = 25^2 = 625 = 18 \cdot 33 + 31 \equiv 31 \equiv -2$

$5^7 = 5^4 \cdot 5^2 \cdot 5 \equiv (-2) \cdot 25 \cdot 5 = -250 \equiv -250 + 264 = \mathbf{14}$

דוגמה 2 · מחתימה להודעה

נתון $n = 33,\ d = 7,\ \sigma = 30$. מה ההודעה? קודם מוצאים את e:

$$\varphi(33) = 2\cdot 10 = 20\ ,\ 3\cdot 7 = 21 \equiv 1 \pmod{20} \Rightarrow e = 3$$

$$m = \sigma^e = 30^3 \equiv (-3)^3 = -27 \equiv \mathbf{6} \pmod{33}$$

דוגמה 3 · מציאת d

נתון $p = 5,\ q = 7,\ e = 11$:

$$n = 35\ ,\ \varphi = 4\cdot 6 = 24$$

$$11\cdot d \equiv 1 \pmod{24} \Rightarrow 11\cdot 11 = 121 = 5\cdot 24 + 1 \Rightarrow d = \mathbf{11}$$

המפתח הפרטי הוא $(11,\ 35)$. d יכול לצאת שווה ל-e. וריאציה: $p = 3,\ q = 17$ נותן $\varphi(51) = 2\cdot 16 = \mathbf{32}$.

דוגמה 4 · איזה זוג (e, d) מתאים?

נתון $n = 33$, והחתימה על $m = 20$ היא 26. $\varphi = 20$, ולכן בודקים קודם אם $e\cdot d \equiv 1 \pmod{20}$:

$$3\cdot 7 = 21 \equiv 1\ \checkmark\ ;\ 3\cdot 11 = 33 \equiv 13\ \times\ ;\ 7\cdot 11 = 77 \equiv 17\ \times\ ;\ 5\cdot 7 = 35 \equiv 15\ \times$$

אימות הזוג $(3,\ 7)$:

$$26^3 \equiv (-7)^3 = -343 \equiv -343 + 363 = 20 = m\ \checkmark$$

דוגמה 5 · RSA עם מספרים גדולים יותר

נתון $p = 13,\ q = 17$, כלומר $n = 221$, ו-$d = 5,\ m = 7$:

$$7^2 = 49\ ,\ 7^4 = 49^2 = 2401 = 10\cdot 221 + 191 \equiv 191 \equiv -30$$

$$\sigma = 7^5 \equiv -30\cdot 7 = -210 \equiv -210 + 221 = \mathbf{11}$$

דוגמה 6 · דיפי-הלמן כשהמספרים הפרטיים נתונים

נתון $n = 23,\ g = 5,\ a = 6,\ b = 15$. מחשבים את B, ואז את המפתח:

$$5^2 = 25 \equiv 2\ ,\ 5^4 \equiv 4\ ,\ 5^8 \equiv 16$$

$$B = 5^{15} = 5^8\cdot 5^4\cdot 5^2\cdot 5 \equiv 16\cdot 4\cdot 2\cdot 5 = 640 = 27\cdot 23 + 19 \equiv 19$$

$$K = B^a = 19^6 \equiv (-4)^6 = 4096 = 178\cdot 23 + 2 \equiv \mathbf{2}$$

דוגמה 7 · דיפי-הלמן כשרק הערכים שנשלחו נתונים

נתון $n = 19,\ g = 3$. אליס שלחה 15 ובוב שלח 5. כותבים את החזקות של 3 מודולו 19 עד שמוצאים את הערכים:

$$3^1{=}3\ ;\ 3^2{=}9\ ;\ 3^3{=}8\ ;\ 3^4{=}\mathbf{5}\ ;\ 3^5{=}\mathbf{15}\ ;\ \Rightarrow a = 5\ ,\ b = 4$$

$$K = B^a = 5^5 : 5^2 \equiv 6\ ,\ 5^4 \equiv 36 \equiv -2\ ,\ 5^5 \equiv -10 \equiv \mathbf{9}$$

$$A^b = 15^4 \equiv (-4)^4 = 256 = 13\cdot 19 + 9 \equiv 9\ \checkmark$$

באותה שיטה: $n = 23,\ g = 3$, נשלחו 12 ו-13, ולכן המפתח **18**. נשלחו 13 ו-4, ולכן המפתח **12**. בהרצאה: $n = 17,\ g = 3$ עם $x = 13,\ 15$ נותנים $y = 12,\ 6$, והמפתח 10.

עוד חישובים שעלולים להופיע

דוגמה 8 · חזקה מהירה (ריבועים חוזרים)

מחשבים $3^{13} \bmod 17$. כותבים את המעריך כסכום חזקות של 2: $13 = 8 + 4 + 1$. מחשבים ריבועים, ומצמצמים אחרי כל שלב:

$$3^1 = 3\ ,\ 3^2 = 9\ ,\ 3^4 = 81 \equiv 13\ ,\ 3^8 \equiv 13^2 = 169 \equiv 16$$

$$3^{13} = 3^8\cdot 3^4\cdot 3^1 \equiv 16\cdot 13\cdot 3 = 624 = 36\cdot 17 + 12 \equiv \mathbf{12}$$

זה בדיוק הערך שאליס שלחה בדוגמת דיפי-הלמן מההרצאה $(g = 3,\ n = 17,\ x = 13)$.

דוגמה 9 · מציאת d כשאין ניחוש קל (אוקלידס המורחב)

נתון $e = 7,\ \varphi = 40$. קודם בודקים ש-$\gcd(7, 40) = 1$, אחרת e לא חוקי (למשל $e = 5$ נפסל כי $\gcd(5, 40) = 5$). מחלקים עד שמגיעים לשארית 1:

$$40 = 5\cdot 7 + 5 \ ; \ 7 = 1\cdot 5 + 2 \ ; \ 5 = 2\cdot 2 + 1$$

ומציבים חזרה מלמטה למעלה:

$$1 = 5 - 2\cdot 2 = 5 - 2\cdot(7 - 5) = 3\cdot 5 - 2\cdot 7 = 3\cdot(40 - 5\cdot 7) - 2\cdot 7 = 3\cdot 40 - 17\cdot 7$$

$$d \equiv -17 \equiv \mathbf{23} \pmod{40} \ ; \ \text{check: } 7\cdot 23 = 161 = 4\cdot 40 + 1 \ \checkmark$$

במבחן אמריקאי לרוב מהיר יותר לבדוק כל תשובה: האם $e\cdot d \bmod \varphi = 1$.

דוגמה 10 · מחזור RSA מלא: הצפנה ופענוח

$p = 5,\ q = 11$, ולכן $\varphi = 40,\ n = 55$, ומדוגמה 9: $e = 7,\ d = 23$. מצפינים $m = 2$:

$$c = 2^7 = 128 = 2\cdot 55 + 18 \equiv \mathbf{18}$$

מפענחים $18^{23} \bmod 55$, כאשר $23 = 16 + 4 + 2 + 1$:

$$18^2 = 324 \equiv 49 \equiv -6 \ , \ 18^4 \equiv 36 \ , \ 18^8 \equiv 36^2 = 1296 \equiv 31 \ , \ 18^{16} \equiv 31^2 = 961 \equiv 26$$

$$18^{23} \equiv 26\cdot 36\cdot(-6)\cdot 18 \ : \ 26\cdot 36 = 936 \equiv 1 \Rightarrow 1\cdot(-6)\cdot 18 = -108 \equiv -108 + 110 = 2 = m \ \checkmark$$

דוגמה 11 · צפנים קלאסיים

Vigenère: מצפינים ATTACK עם המפתח LEMON. משרשרים את המפתח לאורך הטקסט (LEMONL) ומחברים מספרי אותיות מודולו 26 (A=0):

$$\text{A+L} = 0+11 = \text{L} \ ; \ \text{T+E} = 19+4 = \text{X} \ ; \ \text{T+M} = 19+12 = 31 \equiv 5 = \text{F} \ ; \ \text{A+O} = \text{O} \ ; \ \text{C+N} = 2+13 = \text{P} \ ;$$
$$\text{K+L} = 10+11 = \text{V} \Rightarrow \mathbf{\text{LXFOPV}}$$

קיסר בכיוון ההפוך: KHOOR פחות 3 בכל אות נותן **HELLO**. בצופן הזזה כללי יש רק 25 מפתחות, ולכן brute force מסתיים אחרי 25 ניסיונות לכל היותר.

דוגמה 12 · XOR ו-OTP

$$m = 1011 \ , \ k = 0110 \Rightarrow c = m \oplus k = \mathbf{1101} \Rightarrow c \oplus k = 1011 = m$$

אם אותו k משמש פעמיים: $c_1 \oplus c_2 = m_1 \oplus m_2$, כלומר המפתח נעלם לגמרי מהביטוי.

דוגמה 13 · גודל מרחב מפתחות וזמן brute force

- **DES:** מפתח 56 ביט, כלומר $2^{56} \approx 7.2\cdot 10^{16}$ מפתחות. בקצב של 10^9 ניסיונות בשנייה: כ-2.3 שנים במקרה הגרוע, ובממוצע חצי מזה (2^{55} ניסיונות). **AES-128:** 2^{128}, לא בר-ביצוע.
- **סיסמה:** 8 אותיות קטנות באנגלית: $26^8 \approx 2.1\cdot 10^{11}$, כלומר כ-209 שניות ב-$10^9$ hash-ים בשנייה.
- **salt** לא מגדיל את מספר הניסיונות לסיסמה אחת, אבל מכריח לחזור על כל העבודה **לכל משתמש בנפרד** ומבטל טבלאות מוכנות. **hash חוזר i פעמים** מכפיל את עלות כל ניסיון פי i (עם $i = 1000$, 209 שניות הופכות לכ-58 שעות).

דוגמה 14 · פרדוקס יום ההולדת

$$P(\text{collision}) \approx 1 - e^{-k^2/2N} \ ; \ k = 23,\ N = 365 \Rightarrow \approx 50\%$$

- כלל האצבע של הקורס: כ-$\sqrt{N}$ ניסיונות לסיכוי של כ-50%. ל-TXID של 16 ביט: $\sqrt{2^{16}} = \mathbf{256}$. (חישוב מדויק: 256 ניסיונות נותנים כ-39%, ול-50% בדיוק צריך כ-$1.18\cdot\sqrt{N} \approx 302$. במבחן עונים 256.)
- hash של n ביט: התנגשות אחרי כ-$2^{n/2}$ ניסיונות. ל-SHA-256 זה 2^{128}.

דוגמה 15 · הסתברויות בהרעלת DNS

- תשובה מזויפת אחת שמנחשת TXID: $1/2^{16} = 1/65,536$.
- k תשובות מזויפות לאותה שאילתה: בערך $k/2^{16}$. עם k = 100: $100/65,536 \approx 0.15\%$.
- עם פורט מקור אקראי צריך לנחש גם 16 ביט של פורט: $k/2^{32}$, כלומר קשה פי 65,536.

דוגמה 16 · עץ מרקל וביטקוין במספרים

- עץ עם n עלים (n חזקה של 2): $n - 1$ צמתים פנימיים, גובה $\log_2 n$. הוכחת הכללה ושינוי של עלה אחד: $\log_2 n$ צמתים (1024 עלים: 10; 1000 עלים: $10 \to \log_2 1000 \approx 9.97$).
- תקופת קושי: $2016 \times 10 \text{ min} = 20,160 \text{ min} = 14 \text{ days}$.
- פרס בלוק: $50 \to 25 \to 12.5 \to 6.25 \to 3.125$, חצייה כל 210,000 בלוקים. סך כל המטבעות: $210,000 \times 50 \times 2 = 21,000,000$.
- כורה עם 1% מכוח החישוב ימצא בממוצע בלוק אחד מכל 100, כלומר פעם ב-1000 דקות (כ-17 שעות).

דוגמה 17 · בלוקים וריפוד

הודעה של 20 בתים בצופן DES (בלוק של 8 בתים) מתחלקת ל-$20/8 = 2.5 \to 3$ בלוקים, עם 4 בתי ריפוד בבלוק האחרון. ב-ECB, שני בלוקים זהים ב-plaintext נותנים שני בלוקים זהים ב-ciphertext. ב-CBC לא.

סיכום · תשובות מהירות שחוזרות

קיסר על QUIZ	**TXLC**	עץ מרקל עם 1024 עלים	**10** hash-ים
ניחוש TXID בסיכוי 50%	**256**	7 תווים זהים, צופן החלפה על 28 אותיות	**28**
החלפה עם מיפוי אחד ידוע	**25!**	Vigenère, מפתח באורך 4, תווים במרחק 4	**1/26**

12 מילון מושגים

0x20 encoding
אותיות גדולות/קטנות אקראיות בשם הדומיין בשאילתת DNS, שהתשובה חייבת להחזיר בדיוק. מקשה על זיוף.

3DES
שלוש הפעלות DES במבנה E-D-E. עם שלושה מפתחות זהים שקול ל-DES (תמיכה לאחור).

attack 51%
שליטה ברוב כוח הכרייה, שמאפשרת לקבוע אילו עסקאות ייכנסו ולבצע double spending.

AES
צופן בלוקים סימטרי מודרני (בלוק 128 ביט). דורש mode of operation.

Amplification
התקפת DDoS שבה התשובה גדולה מהבקשה (למשל DNS דרך open resolver), כך שהתוקף מגביר את הנזק.

Anti-CSRF token
ערך אקראי בטופס שהשרת בודק. שקוף למשתמש. נשבר אם יש XSS.

ARP
פרוטוקול שממפה IP ל-MAC ברשת מקומית. אין בו אימות, והתשובה האחרונה דורסת.

ARP poisoning
שליחת תשובות ARP שקריות כדי שתעבורה תגיע לתוקף. מספיק off-path, ומשיג MitM.

Bailiwick rule
שרת DNS רשאי לספק רשומות רק לתחום שבסמכותו. רשומות זרות נזרקות.

Birthday paradox
כ-√N ניסיונות נותנים 50% להתנגשות מתוך N ערכים (23 אנשים ליום הולדת משותף).

Blacklist / Whitelist
רשימה שחורה: ברירת מחדל לאפשר וחוסמים את הרשום. רשימה לבנה: ברירת מחדל לחסום ומאפשרים רק את הרשום.

Blind SQL injection
הזרקה בלי לראות פלט, תוך הסקת אמת/שקר מהתנהגות הדף או מזמן התגובה.

Block cipher
צופן שמצפין בלוקים בגודל קבוע (DES, AES).

Botnet / C&C
רשת מחשבים שנפרצו, שנשלטת דרך שרתי פיקוד ושליטה. מפילים את ה-C&C כדי לנטרל.

Brute force
חיפוש ממצה של כל המפתחות או הסיסמאות האפשריים.

CA
רשות מאשרת שמוודאת בעלות על דומיין וחותמת על תעודות במפתח הפרטי שלה.

CBC
Mode שבו כל בלוק עובר XOR עם ה-ciphertext הקודם לפני ההצפנה. מתחיל ב-IV אקראי.

Certificate Transparency (CT)
רישום חובה של כל תעודה ביומנים ציבוריים (עץ מרקל), לגילוי הנפקה זדונית.

Chain of trust
שרשרת חתימות מתעודת האתר דרך intermediate CA עד Root CA שבמאגר האמון.

Clickjacking
iframe שקוף שגורם ללחיצה לא מודעת. הגנה: X-Frame-Options / frame-ancestors.

Cookie / HttpOnly
מחרוזת שהדפדפן מצרף לבקשות לאתר. HttpOnly חוסם גישה אליה מ-JavaScript.

CRL / CRLSets
רשימת תעודות שבוטלו שה-CA מפרסם / רשימה מצומצמת שהדפדפן מוריד.

CSP
Header שמגדיר מאילו מקורות מותר לדפדפן לטעון ולהריץ משאבים. הגנה מ-XSS.

CSRF / XSRF
זיוף בקשה מדפדפן הקורבן לאתר שבו הוא מחובר, תוך ניצול צירוף אוטומטי של עוגיות.

DDoS
מניעת שירות ממקורות רבים (botnet).

DES
צופן בלוקים סימטרי ישן: בלוק 64 ביט, מפתח 56 ביט.

Diffie-Hellman
הסכמה על מפתח משותף בערוץ גלוי, על בסיס בעיית הלוגריתם הדיסקרטי. פגיע ל-MitM.

Digital signature
חתימה בפרטי ואימות בפומבי. נותנת אמינות, אותנטיות ואי-הכחשה.

DMZ
תת-רשת מופרדת לשרתים שנגישים מבחוץ.

DNS poisoning
שתילת רשומת DNS שקרית ב-cache של resolver.

DNSSEC
חתימה דיגיטלית על רשומות DNS. מונעת זיוף תשובות.

Double spending
שימוש באותו מטבע דיגיטלי פעמיים.

Downgrade (שנמוך)
אילוץ שימוש בגרסה או אלגוריתם חלשים (switch ל-hub, TLS לצופן חלש, HTTPS ל-HTTP).

ECB
Mode שמצפין כל בלוק בנפרד. בלוקים זהים מוצפנים זהה.

Encrypt-then-MAC
מצפינים ואז מחשבים MAC על ה-ciphertext. המקבל בודק MAC לפני פענוח.

Escrow
עסקת נאמנות: ארנק 2-מתוך-3 של קונה, מוכר וצד שלישי.

Fork (soft / hard)
פיצול בשרשרת. Soft: כללים מוחמרים, תואם לאחור. Hard: שינוי שצמתים ישנים דוחים.

Forward secrecy
גניבת מפתח ארוך טווח לא חושפת תעבורה מהעבר. אין כזו ב-RSA key exchange.

Hash function
פונקציה חד-כיוונית לפלט קבוע (SHA-256). תכונות: pre-image, second pre-image, collision.

HSTS / Preload
הוראה לדפדפן לגשת לאתר רק ב-HTTPS / רשימה מובנית בדפדפן גם לביקור ראשון.

Hybrid encryption
מפתח סימטרי שמועבר מוצפן במפתח פומבי.

IDS / IPS
מערכת זיהוי חדירות (פסיבית, מתריעה) / מערכת מניעה (חוסמת).

IV
וקטור אתחול אקראי, לא סודי, לבלוק הראשון ב-CBC.

Kaminsky attack
הרעלת DNS דרך שאלות על תתי-דומיינים לא קיימים, שעוקפת את ה-cache.

Kerckhoffs
הבטיחות צריכה להישען רק על סודיות המפתח.

LockTime
שדה שמונע מעסקה להיכנס לבלוק לפני זמן מסוים (micropayments).

Logger / Monitor / SCT
יומן CT ציבורי / מי שמנטר את הלוגרים / חותמת זמן חתומה של הלוגר שמוטמעת בתעודה.

MAC (קוד)
Tag שמחושב עם מפתח משותף. אמינות ואותנטיות, בלי אי-הכחשה.

MAC flooding
הצפת טבלת ה-CAM של switch כך שיתנהג כמו hub. הגנה: port security.

Merkle tree
עץ hash-ים בינארי. הוכחת הכללה ב-log(n) ערכים.

Micropayments
תשלומים קטנים רבים, שרק האחרון מתפרסם, עם החזר מובטח דרך LockTime.

Mining pool
קבוצת כורים שמחלקת עבודה ופרס.

MitM
תוקף בתווך: קורא, משנה, חוסם ומזריק.

Mode of operation
שיטה להצפנת הודעה ארוכה בצופן בלוקים (ECB, CBC).

Multisig
כתובת שדורשת כמה חתימות כדי לבזבז.

Nonce (ביטקוין)
מספר שמשנים בחיפוש אחר hash של בלוק עם מספיק אפסים.

Non-repudiation
אי-הכחשה: השולח לא יכול להכחיש. ניתנת ע"י חתימות, לא ע"י MAC.

OCSP / Stapling
שאילתת סטטוס תעודה בזמן אמת / השרת מצרף תשובה חתומה מראש.

Off-path
תוקף שיכול רק לשלוח הודעות (כולל מזויפות). לא רואה ולא חוסם.

One-Time Pad
XOR עם מפתח אקראי באורך ההודעה, לשימוש חד-פעמי. בטוח לחלוטין.

Origin
פרוטוקול + דומיין + פורט.

Pepper
סוד שמשורשר לסיסמה ולא נשמר ב-DB.

PKI
תשתית שקושרת מפתחות פומביים לזהויות באמצעות תעודות ו-CA.

Port security
הגבלת כתובות MAC לכל פורט ב-switch.

Prepared statement
שאילתה עם פרמטרים, שבה הקלט לא משנה את המבנה. ההגנה העיקרית מ-SQLi.

Proof of Work / Stake
זכות יצירת בלוק לפי עבודה חישובית / לפי כמות מטבעות.

PRG
מחולל פסאודו-אקראי שמרחיב seed קצר לרצף ארוך.

Reflection (TLS)
החזרת הודעה של הלקוח אליו כאילו מהשרת. נמנעת עם מפתח נפרד לכל כיוון.

Root store
רשימת Root CA שמערכת ההפעלה או הדפדפן סומכים עליהם.

RSA
הצפנה וחתימה א-סימטריות, על בסיס קושי פירוק $n = p \cdot q$.

Salt
ערך אקראי לכל משתמש שמשורשר לסיסמה ונשמר גלוי. מונע מילונים מוכנים.

Same-Origin Policy
מנגנון בדפדפן שמונע מסקריפט לקרוא מידע של origin אחר.

SameSite cookie
עוגייה שלא נשלחת בבקשות שמקורן באתר אחר. הגנה מ-CSRF.

Session resumption
חידוש חיבור TLS עם session id קודם, בלי ClientKeyExchange.

Slowloris
DoS בקצב נמוך שמחזיק חיבורי HTTP פתוחים עם בקשות חלקיות.

Soft fail
קבלת תעודה כשאין תשובת ביטול. ניתן לניצול ע"י MitM.

Spoofing
זיוף כתובת מקור (IP, MAC) או זהות.

SQL injection
שינוי מבנה שאילתת SQL דרך קלט משתמש.

Stateful / Stateless filter
חומת אש שזוכרת חיבורים / שמחליטה על כל חבילה בנפרד.

Stream cipher
XOR עם רצף פסאודו-אקראי (RC4). מהיר.

SYN cookies
קידוד מצב החיבור במספר הסידורי, בלי זיכרון בשרת. הגנה מ-SYN flood.

SYN flood
הצפת SYN מכתובות מזויפות כדי למלא את זיכרון החיבורים החצי-פתוחים.

TLS
פרוטוקול להצפנת חיבור: handshake א-סימטרי ותעבורה סימטרית.

TOFU
Trust On First Use: הביקור הראשון חשוף (הבעיה של HSTS בלי preload).

TOR / Mix net
ניתוב בשכבות הצפנה דרך 3 צמתים / ערבוב והשהיית הודעות.

Transaction ID
מזהה של 16 ביט לשאילתת DNS.

VPN
מנהרה מוצפנת לשרת שגולש בשמך.

Vigenère
צופן הזזה עם מפתח חוזר. נשבר ע"י מציאת אורך המפתח וניתוח שכיחויות.

WAF
חומת אש לאפליקציות Web, שבודקת תוכן HTTP.

XSS
הזרקת סקריפט שרץ בדפדפן הקורבן מתוך האתר הפגיע. סוגים: stored, reflected, DOM-based.

מקורות

הסיכום מבוסס על סיכום ההרצאות של הקורס (הרצאות 1–12), על הסילבוס ועל 26 קבצי המבחנים שבמאגר (2022–2026). השלמות ואימותים מהאינטרנט:

- TLS 1.3 / RFC 8446, הסרת RSA key exchange ו-forward secrecy: noze.it/en/insights/tls-1-3-rfc-8446
- מדיניות Certificate Transparency של Chrome (מספר SCT-ים): groups.google.com/a/chromium.org/g/ct-policy
- OCSP Stapling, soft-fail ו-Must-Staple: ssl.com/article/ocsp-stapling-secure-and-efficient-certificate-validation
- חצייית פרס וכוונן קושי בביטקוין: forklog.com/en/what-is-the-bitcoin-halving
- SameSite cookies ו-CSRF: simonwillison.net/2021/Aug/3/samesite